Ollscoil Teicneolaíochta an Atlantaigh

Atlantic Technological University

Assignment No: 4

Date: 28th October 2026

Title: Unipolar and Polar NRZ and RZ Line Codes using Python and Simulink

Name: David Roberto Tapia Caicedo

Course: Bachelor of Engineering in Electronic Engineering

Subject: Digital Communications and Transmission

Lecturer: Ben Graham

Summary: To generate and investigate various line codes (NRZ, RZ) using python and simulink

OBJECTIVES:
After the completion of this experiment, students will have good knowledge about the four line codes:

- Unipolar NRZ
- Polar NRZ
- Unipolar RZ
- Polar RZ

EQUIPMENT REQUIRED:
Computer with access to Kaggle.

THEORY:

Unipolar NRZ	The signal stays at +V for the entire bit interval for a binary one and 0V for the total bit time for a binary zero.
Polar NRZ	The signal stays at +V for the entire bit interval for a binary one and -V for the total bit time for a binary zero.
Unipolar RZ	The signal stays at +V for 50% and 0V for the other 50% of the bit interval for a binary one and 0V for the total bit time for a binary zero.
Polar RZ	The signal stays at +V for 50% and 0V for the other 50% of the bit interval for a binary one and -V for 50% and 0V for the other 50% of the bit time for a binary zero.

Total Marks : 100

All work must be completed and submitted inside THIS Document

Part 1- Unipolar NRZ - (Marks:20)

a) Register an account on Kaggle using your student email address.

b) Once you log onto your Kaggle account, on the left hand side, select the code icon.

c) This will open a Kaggle notepad and allow us to write python based code to simulate line codes.

d) The code in Figure 1 below allows us to create a simple NRZ waveform for a given binary code.

```python
import numpy as np
import matplotlib.pyplot as plt

# Parameters
V_high = 1        # Voltage for bit 1
V_low = 0         # Voltage for bit 0
bit_duration = 1  # Duration of one bit

bits=[1,0,1,0,1,0,1]

# Step 2: Create NRZ waveform
t = []
y = []

for i, bit in enumerate(bits):
    t_bit = np.arange(i*bit_duration, (i+1)*bit_duration, 0.01)
    y_bit = np.ones_like(t_bit) * (V_high if bit == 1 else V_low)
    t.extend(t_bit)
    y.extend(y_bit)

# Step 3: Plot NRZ waveform
plt.figure(figsize=(10,3))
plt.plot(t, y, linewidth=2)
plt.ylim(-0.5, +1.5)
plt.xlabel("Time")
plt.ylabel("Voltage")
plt.title(f"Unipolar NRZ Example'")
plt.grid(True)
plt.show()
```

Figure 1: Base NRZ code

Task 1: Base NRZ Implementation

Using the core setup logic for a standard binary line code simulation, execute a Python script to generate a basic Unipolar NRZ waveform.

Answer 1: Paste the waveform you have created here and comment on it.

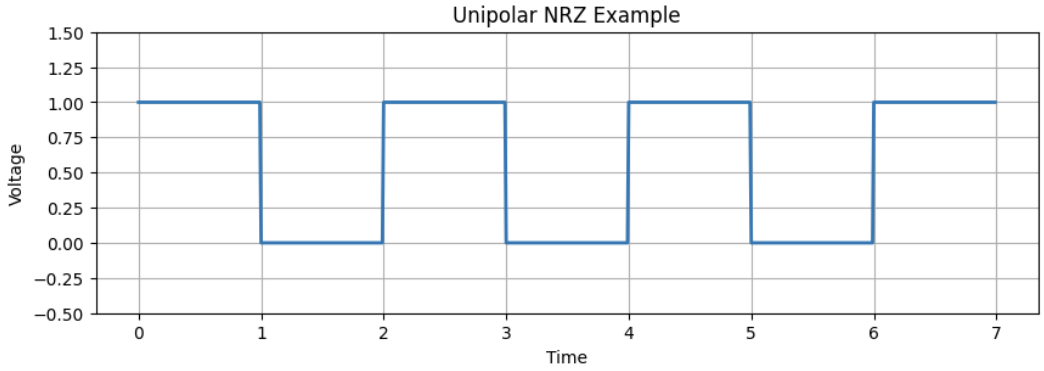


Figure 2: 1010101 Unipolar NRZ Waveform

It is a 7-bit sequence (1010101), where the high voltage level is at 1V and the low voltage level is at 0V. The timing is determined by sequences of 1-bit periods.
It is a unipolar NRZ waveform, which means that it uses only one polarity, the positive polarity in this case. Furthermore, it does not return to 0 volts unless there is a falling edge.

Task 2: Custom Sequence Generation (+5V)

Change the program to generate a +5 V Unipolar NRZ waveform for the 8-bits, 11010010.

Answer 2 : Paste the regenerated waveform you have created here and comment on it.

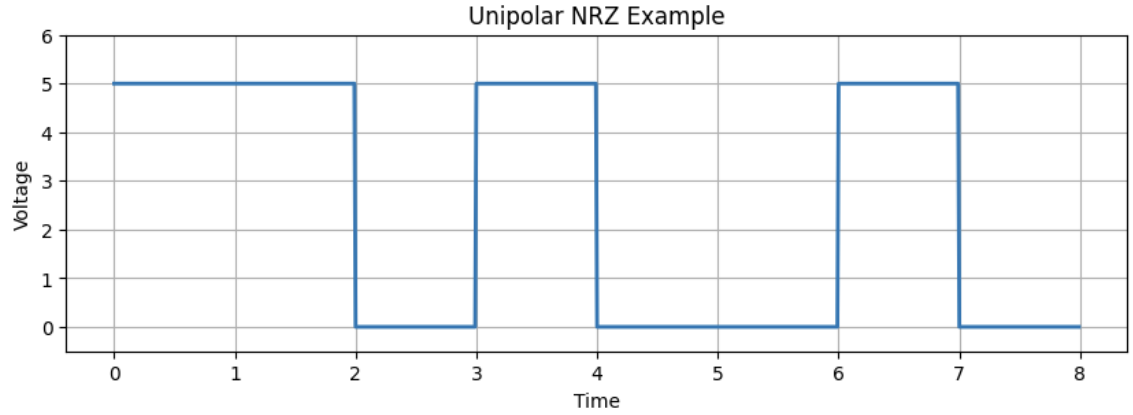


Figure 3: 11010010 Unipolar NRZ Waveform

It is an 8-bit sequence (11010010) in which the high voltage level is at +5 V and the low voltage level is at 0 V. As in the previous exercise, the time is divided into 1-bit intervals. Similarly, it is a unipolar NRZ waveform.

Task 3: Character Encoding Generation (+5V)

Change the program to generate a +3.3V Unipolar NRZ waveform for:ASCII for first letter of your name

---

Answer 3 : Paste the generated waveform you have created here and comment on it.

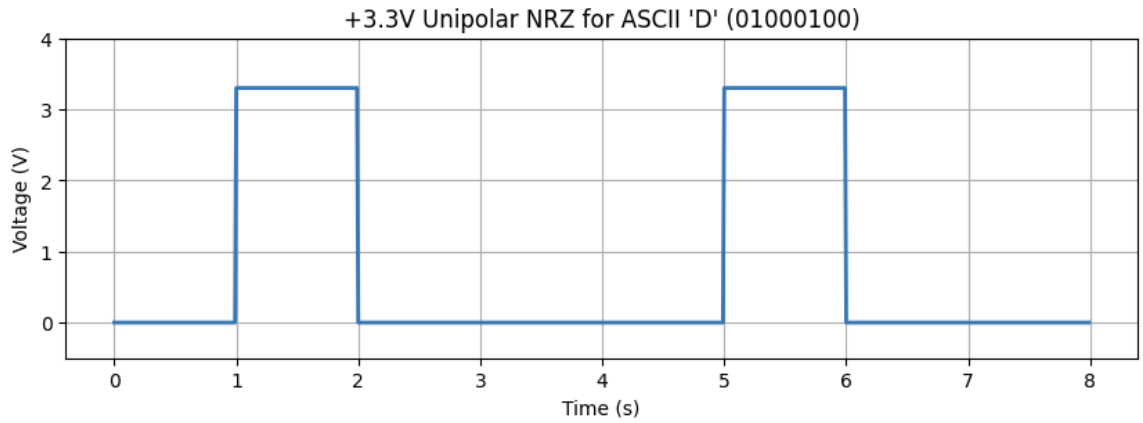


Figure 4: 'D' Unipolar NRZ Waveform

This waveform represents an 8-bit sequence (01000100), which corresponds to the first letter of my name, D, in David. It has the high voltage level at 3.3 V and the low voltage level at 0 V. It can be observed that this character goes to 3.3 V at bits 1 and 5.

Hint Code:

```
#char = 'B'       # ASCII character to convert
# character to binary (8-bit)
#ascii_code = ord(char)
#binary_str = format(ascii_code, '08b')
#bits = [int(b) for b in binary_str]
```

Note: Google/AI will help with python functions/ideas.

Part 2 – Polar NRZ - (Marks:10)

Edit your existing code

Task 4: Character Encoding Generation (+/-12V)

Generate a ±12 V Polar NRZ line code for ASCII '#'.

Answer 4: Copy the code and waveform and paste here ensure waveform is labelled and has a corresponding title.

Code:

```
import numpy as np
import matplotlib.pyplot as plt
V_high = 12
V_low = -12
bit_duration = 1

char = '#'
ascii_code = ord(char)
binary_str = format(ascii_code, '08b')
bits = [int(b) for b in binary_str]

t, y = [], []
for i, bit in enumerate(bits):
    t_bit = np.arange(i*bit_duration, (i+1)*bit_duration, 0.01)
    y_bit = np.ones_like(t_bit) * (V_high if bit == 1 else V_low)
```

```
    t.extend(t_bit)
    y.extend(y_bit)

plt.figure(figsize=(10,3))
plt.plot(t, y, linewidth=2)
plt.ylim(-15, 15)
plt.xlabel("Time (s)")
plt.ylabel("Voltage (V)")
plt.title(f"±12V Polar NRZ for ASCII '{char}' ({binary_str})")
plt.grid(True)
plt.show()
```

```
Character: #, ASCII: 35, Binary: 00100011
```

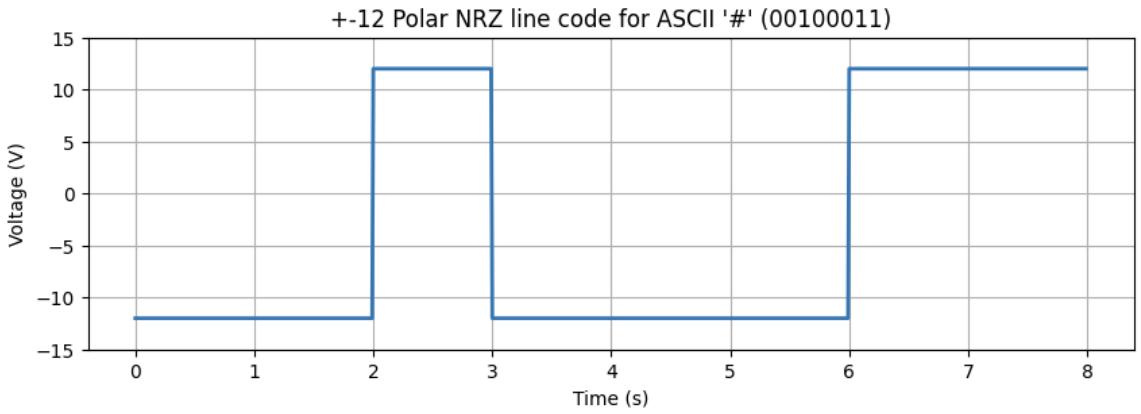


Figure 5: '#' Polar NRZ Waveform

This waveform represents an 8-bit sequence (00100011), with the high voltage level at +12 V and the low voltage level at -12 V. This type of signal is characterized by using both polarities to represent the rising and falling edges, as can be seen in this case.

Part 3 – UniPolar RZ - (Marks:15)

Using the code below in Figure 6

Task 5: Character Encoding Generation

Generate a +3 V Unipolar RZ line code for 0xB3 hex.

Hint:

```
if bit == 1:
    # High for first half of the bit
    t_high = np.arange(i*bit_duration, i*bit_duration + bit_duration/2, 0.01)
    y_high = np.ones_like(t_high) * V_high
    # Return to zero for second half
    t_low = np.arange(i*bit_duration + bit_duration/2, (i+1)*bit_duration, 0.01)
    y_low = np.zeros_like(t_low)
    # Append
    t.extend(t_high)
    t.extend(t_low)
    y.extend(y_high)
    y.extend(y_low)
else:
    # Bit 0 stays low for full bit
    t_bit = np.arange(i*bit_duration, (i+1)*bit_duration, 0.01)
    y_bit = np.zeros_like(t_bit)
    t.extend(t_bit)
```

Figure 6: Base RZ code

Answer 5: Copy the code with waveform and paste here ensure waveform is labelled and has a corresponding title.

Code

```
import numpy as np
import matplotlib.pyplot as plt
V_high = 3
bit_duration = 1

hex_val = 0xB3
binary_str = format(hex_val, '08b') # '10110011'
bits = [int(b) for b in binary_str]

t, y = [], []
for i, bit in enumerate(bits):
    if bit == 1:
        # High for first half of the bit
        t_high = np.arange(i*bit_duration, i*bit_duration + bit_duration/2, 0.01)
        y_high = np.ones_like(t_high) * V_high
        # Return to zero for second half
        t_low = np.arange(i*bit_duration + bit_duration/2, (i+1)*bit_duration,
0.01)
        y_low = np.zeros_like(t_low)
        t.extend(t_high)
        t.extend(t_low)
        y.extend(y_high)
        y.extend(y_low)
    else:
        # Bit 0 stays low for full bit
        t_bit = np.arange(i*bit_duration, (i+1)*bit_duration, 0.01)
        y_bit = np.zeros_like(t_bit)
```

```
        t.extend(t_bit)
        y.extend(y_bit)

plt.figure(figsize=(10,3))
plt.plot(t, y, linewidth=2)
plt.ylim(-0.5, 4)
plt.xlabel("Time (s)")
plt.ylabel("Voltage (V)")
plt.title(f"+3V Unipolar RZ for Hex 0xB3 ({binary_str})")
plt.grid(True)
plt.show()
```

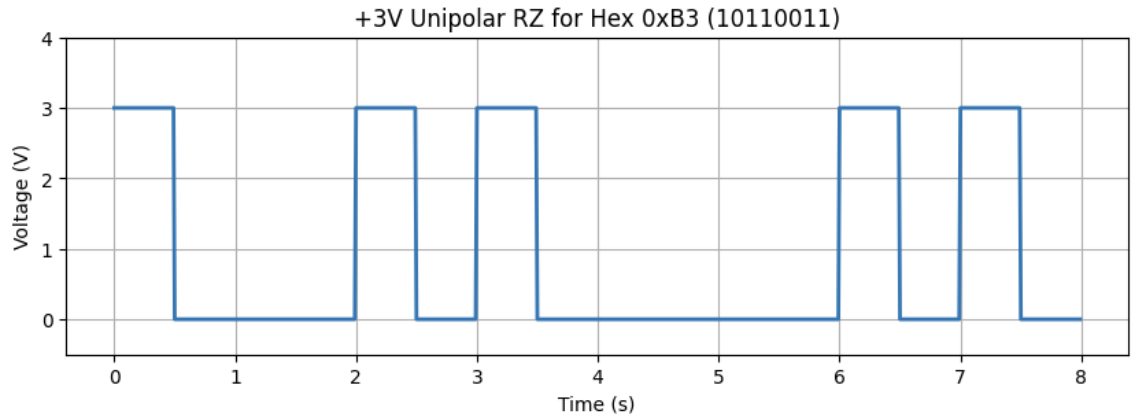


Figure 7: '0xB3' Unipolar RZ Waveform

In this 8-bit waveform (10110011), the high voltage level is at +3 V and the low voltage level is at 0V, since it is unipolar. Furthermore, it uses RZ encoding, meaning that the signal returns to 0V halfway through each bit period.

Part 4 – Polar RZ - (Marks:15)

Edit your existing code to

Task 6: Character Encoding Generation

Generate a ±3 V Polar RZ line code.

Answer 6: Copy the code with waveform and paste here ensure waveform is labelled and has a corresponding title.

Code:

```
import numpy as np
import matplotlib.pyplot as plt
V_high = 3
V_low = -3
bit_duration = 1

hex_val = 0xB3
binary_str = format(hex_val, '08b')
bits = [int(b) for b in binary_str]

t, y = [], []
for i, bit in enumerate(bits):
    if bit == 1:
        t_high = np.arange(i*bit_duration, i*bit_duration + bit_duration/2, 0.01)
        y_high = np.ones_like(t_high) * V_high
```

```
        t_zero = np.arange(i*bit_duration + bit_duration/2, (i+1)*bit_duration,
0.01)
        y_zero = np.zeros_like(t_zero)
        t.extend(t_high)
        t.extend(t_zero)
        y.extend(y_high)
        y.extend(y_zero)
    else:
        # Bit 0 goes to -3V for first half, then returns to 0V
        t_low = np.arange(i*bit_duration, i*bit_duration + bit_duration/2, 0.01)
        y_low = np.ones_like(t_low) * V_low
        t_zero = np.arange(i*bit_duration + bit_duration/2, (i+1)*bit_duration,
0.01)
        y_zero = np.zeros_like(t_zero)
        t.extend(t_low)
        t.extend(t_zero)
        y.extend(y_low)
        y.extend(y_zero)

plt.figure(figsize=(10,3))
plt.plot(t, y, linewidth=2)
plt.ylim(-4, 4)
plt.xlabel("Time (s)")
plt.ylabel("Voltage (V)")
plt.title(f"±3V Polar RZ for Hex 0xB3 ({binary_str})")
plt.grid(True)
plt.show()
```

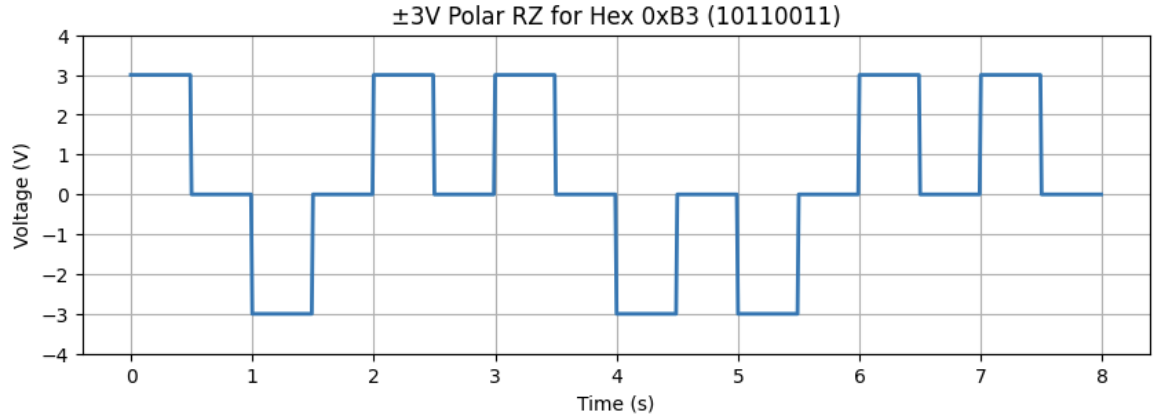


Figure 8: '0xB3' Polar RZ Waveform

This waveform represents the 8-bit sequence for 0xB3 (10110011) using +-3V Polar RZ coding. Here, binary 1 goes to +3V for the first half of the bit period and returns to 0V, while binary 0 goes to -3V for the first half before returning to 0V. Having a transition back to zero in the middle of every single bit helps the receiver maintain clock synchronisation.

Part 5 – Simulink - (Marks:40)

Move now to Matlab on line (you should have an account from last year), open Simulink with Matlab.

Task 7: Unipolar NRZ

a) Generate a +5 V Unipolar NRZ line code using the following blocks:

First place a binary signal source, the binary signal source will randomly outputting either a logic 0 or 1.
As Simulink basic is the only option available to us, obtaining a simple logic generator is more involved than it should be and requires the following 3 components

- Uniform Random Number
- Relational Operator
- Constant

The uniform random generator settings at Min = 0, Max = 1, Seed = 1 and sample time 0.5. The constant is setup to output a constant 0.5. The relational operator set to greater or equal (>=).

This will create a random logic output between 0 and 1.

Next create the 5V unipolar NRZ output. The signal will stay at the +5V for a binary 1 and 0V for a binary 0.

To achieve this an additional component is required

- Gain block (set to 5)

Set simulation time to 10 seconds, scope the output of the relational operator and the gain block to view the input logic and output unipolar NRZ.

Answer 7: Include Simulink circuit and waveforms here.

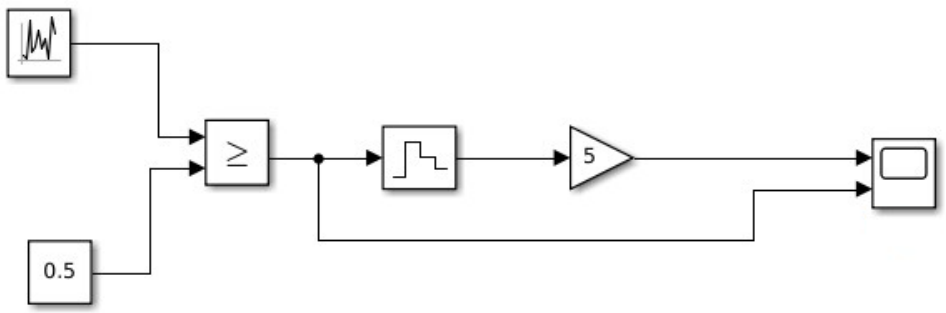


Figure 9: Exercise 7 Simulink circuit

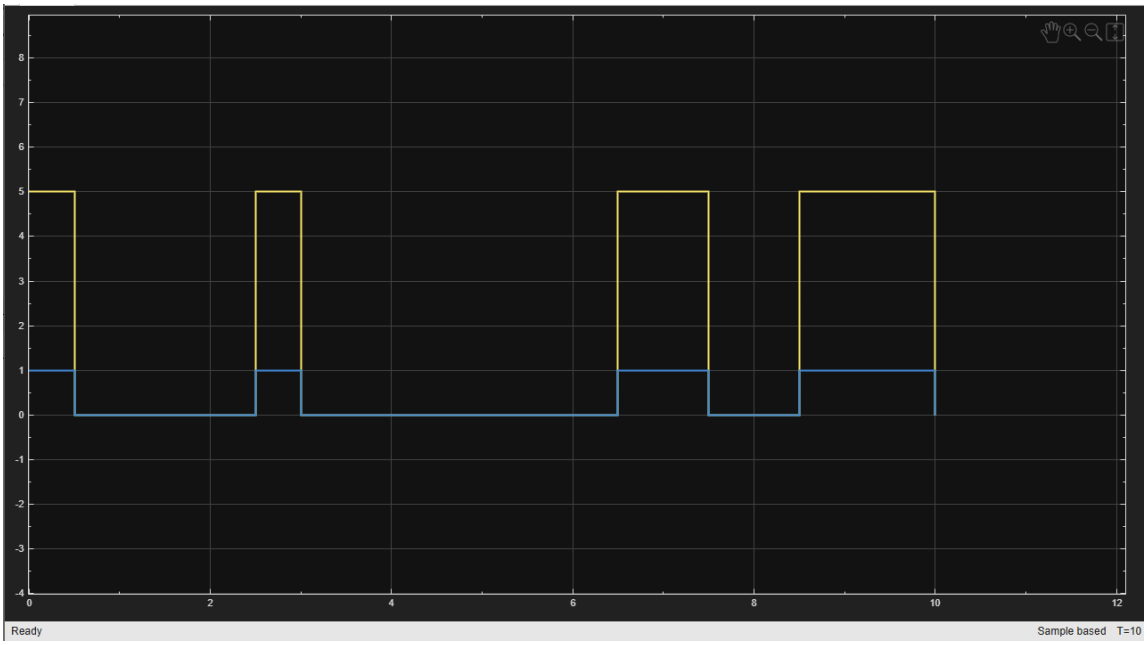


Figure 10: 5V unipolar NRZ output

As shown in Figure 9 and Figure 10, the random binary logic signal represented by blue trace, is amplified by a gain of 5 to produce a +5V Unipolar NRZ waveform, the yellow trace, where logic 1 stays at +5V and logic 0 stays at 0V.

Task 8: Polar NRZ

b) Generate a +5 V Polar NRZ line code:

Next create the 5V polar NRZ output. The signal will stay at the +5V for a binary 1 and -5V for a binary 0.

To achieve this 3 additional components are required

- Gain block set to 2
- Sum block
- Constant set to - 1

The logic output 0 or 1 needs to be amplified by 2, this output is this added to -1. We get the +/- 5V output required

$$Bit\ 0 = (2 \times 0) = 0 + (-1) = -1$$
$$Bit\ 1 = (2 \times 1) = 2 + (-1) = -1$$

The output of this sum is then multiplied by 5, to obtain the +5V.

Answer 8: Include Simulink circuit and waveforms here.

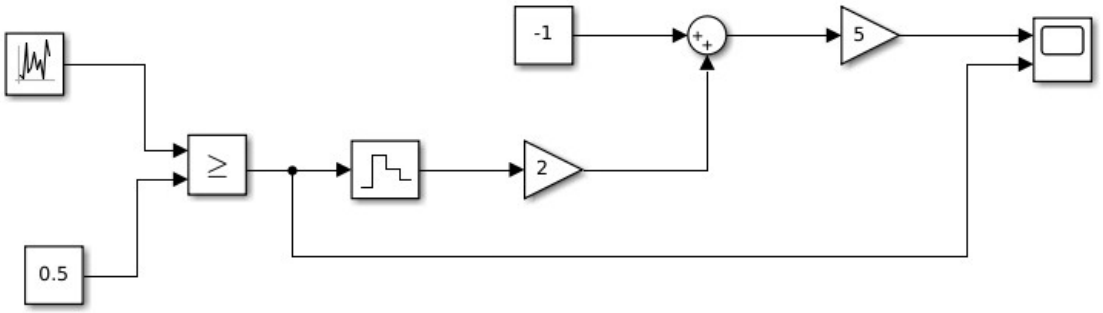


Figure 11: Exercise 8 Simulink circuit

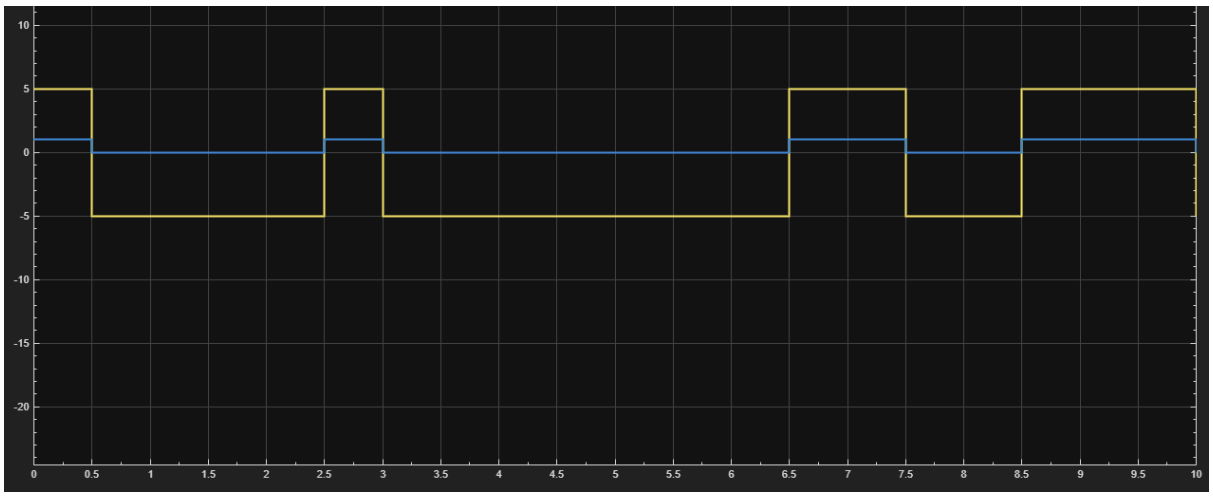


Figure 12: 5V polar NRZ output

In Figure 11 and Figure 12, the unipolar logic input, blue trace, is multiplied by 2, shifted by -1, and amplified by a gain of 5. This generates the +-5V Polar NRZ output (yellow trace), which stays at +5V for a binary 1 and -5V for a binary 0.

Task 9: Polar RZ

c) Generate a +5 V Unipolar RZ line code using the following blocks:

Once again a logic signal is needed for all of the following sections, this randomly outputting either a 0 or 1. We will use the same circuit as before for this section to create the 0 and 1 bits logic input.

In additional for the RZ the waveform needs to return to 0 for the last half of the positive. This can be achieved by using

- Pulse Generator
- Product

The pulse generator is set to the same frequency as the random generator (0.5 seconds). Duty cycle is set to 50%.

The output of the pulse generator and the logic signal (zero order hold) can then be fed to product. Output will be RZ compliant, gain, etc will be required to achieve the correct voltages.

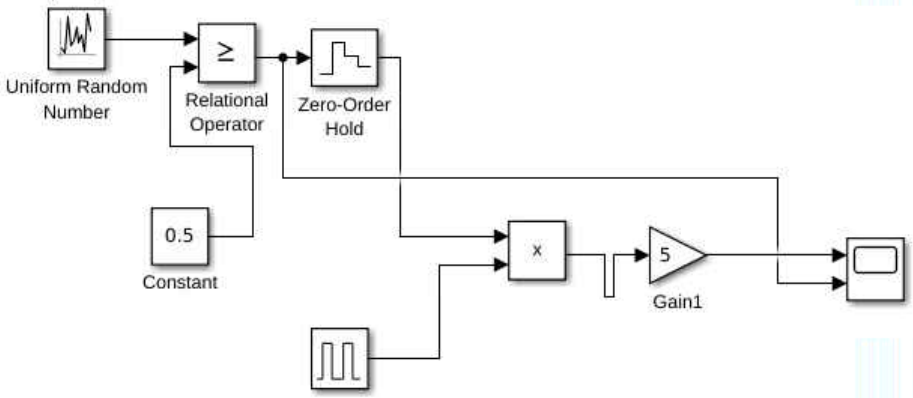


Figure 13: Simulink RZ unipolar

Answer 9: Include Simulink circuit and waveforms in here

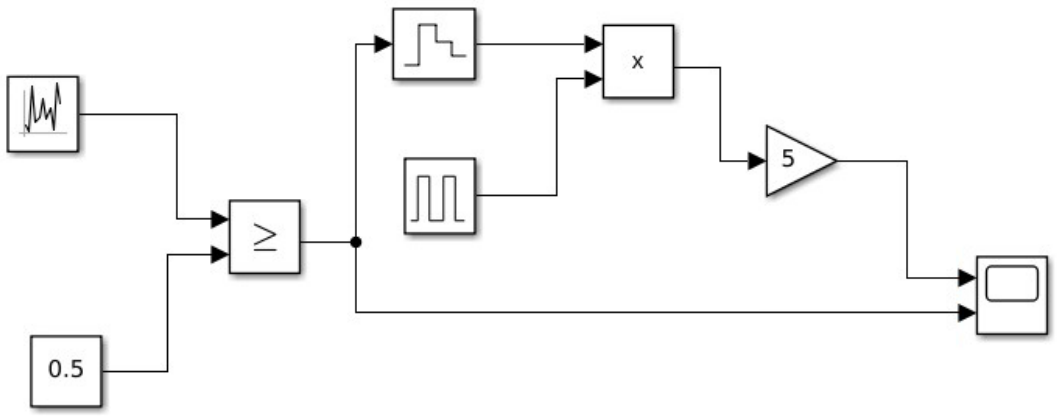


Figure 14: Exercise 9 Simulink circuit

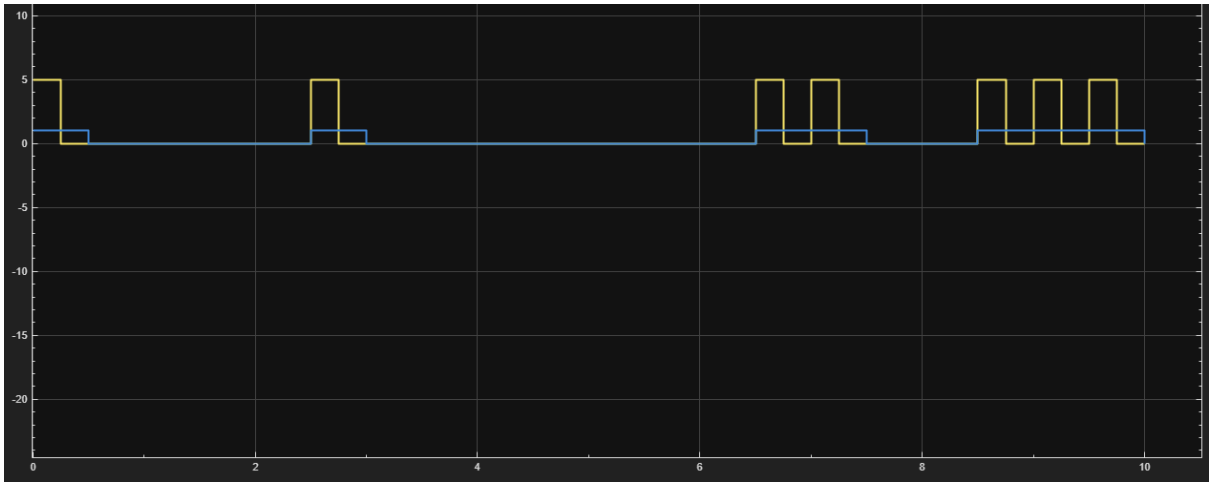


Figure 15: +5 V Unipolar RZ output

Figure 14 and Figure 15 show the +5V Unipolar RZ simulation. By multiplying the held logic signal with a 0.5s pulse generator before the gain of 5, the yellow output trace goes to +5V for the first half of a logic 1 and returns to 0V for the second half, while remaining at 0V during logic 0.

Task 10: Polar RZ

d) Generate a +3 V Polar RZ line code:

Repeat for +3V polar RZ line code

Answer 10: Include Simulink circuit and waveforms here

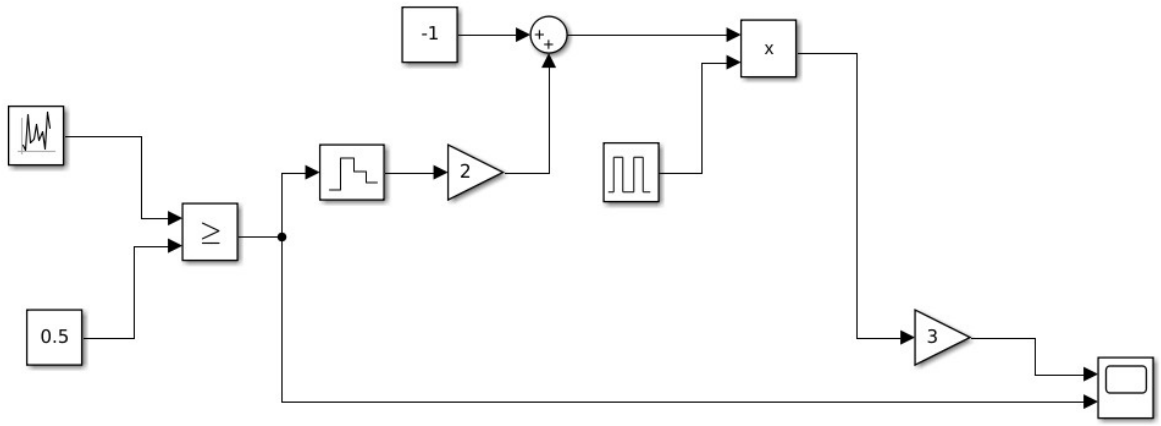


Figure 16: Exercise 10 Simulink circuit

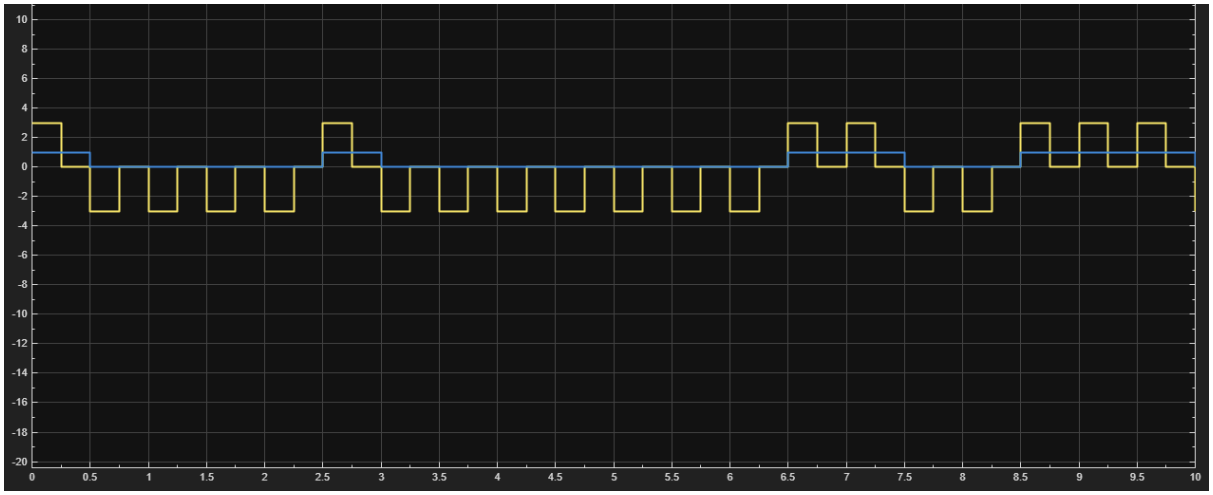


Figure 17: +3V polar RZ output

Finally, Figure 16 and Figure 17 combine the polar conversion stage with the pulse multiplier and a gain block of 3. The resulting +-3V Polar RZ waveform, the yellow trace, pulses to +3V for the first half of a binary 1 and to -3V for the first half of a binary 0, returning to 0V halfway through every 0.5s bit interval.

Note: Ensure all questions are answered and included in the worksheet